

Compilador HULK: Manual de Usuario y Documentación Técnica

David Sánchez

July 4, 2025

Abstract

Este documento presenta una guía de usuario y documentación técnica del compilador para el lenguaje de programación HULK. Se describen los aspectos esenciales para utilizarlo, desde la instalación hasta la ejecución de programas, así como una visión general del diseño e implementación del compilador, incluyendo el análisis léxico, sintáctico, semántico y la generación de código.

Contents

1	Introducción	3
2	Instalación y Compilación	3
2.1	Requisitos Previos	3
2.2	Comandos del Makefile	3
2.3	Compilar y Ejecutar un Programa HULK	3
3	Sintaxis del Lenguaje HULK	3
3.1	Estructura General	3
3.2	Tipos de Datos	4
4	Arquitectura del Compilador	4
4.1	Análisis Léxico con Flex	4
4.1.1	Funcionamiento de Flex	4
4.2	Análisis Sintáctico con Bison	4
4.2.1	Funcionamiento de Bison	4
4.3	Árbol de Sintaxis Abstracta (AST)	5
4.4	Análisis Semántico	5
4.4.1	Recolector de Tipos	5
4.4.2	Recolector de Símbolos	5
4.4.3	Verificador Semántico	5
4.5	Generación de Código	5
4.5.1	LLVM y JIT	5
5	Ejemplos de Código	5
5.1	Hola Mundo	5
5.2	Declaración de Variables y Expresiones	6
5.3	Definición de Funciones	6
5.4	Estructuras Condicionales	6
5.5	Definición de Tipos	6
6	Resolución de Problemas	6
6.1	Errores Comunes	6
7	Conclusión	7

8	Implementaciones Alternativas	7
8.1	Lexer Personalizado	7
8.1.1	Características Principales	7
8.1.2	Implementación	7
8.2	Parser SLR1 Personalizado	8
8.2.1	Características Principales	8
8.2.2	Implementación	8
8.2.3	Estado Actual de la Implementación	8
A	Gramática Completa de HULK en Bison	9
B	Notas sobre la Gramática	12

1 Introducción

HULK (Havana University Language for Kompilers) es un lenguaje de programación desarrollado con fines educativos. Este documento proporciona instrucciones sobre cómo compilar y utilizar el compilador de HULK, así como una descripción técnica de sus componentes principales.

2 Instalación y Compilación

Para utilizar el compilador, necesitarás tener Flex y Bison instalados, así como el compilador de C++ *clang++*. El proyecto se compila utilizando un ‘makefile’.

2.1 Requisitos Previos

- Flex 2.6 o superior
- Bison 3.0 o superior
- Clang++ 17 o superior
- LLVM 10.0 o superior

2.2 Comandos del Makefile

El ‘makefile’ proporcionado automatiza las tareas de compilación y limpieza. Los principales comandos son:

- **Limpiar el proyecto (‘make clean’)**: Elimina todos los ficheros objeto (‘.o’) y el ejecutable final. Es útil para realizar una compilación desde cero.

```
make clean
```

- **Compilar el proyecto (‘make compile’)**: Compila todo el código fuente y genera los artefactos necesarios en la carpeta ‘hulk/’.

```
make compile
```

- **Ejecutar el compilador (‘make execute’)**: Este comando ejecuta el compilador. Si los artefactos no existen o el código fuente ha cambiado, primero compilará el proyecto y luego lo ejecutará.

```
make execute
```

2.3 Compilar y Ejecutar un Programa HULK

Una vez que el compilador ‘hulk’ ha sido generado, puedes usarlo para compilar y ejecutar tus propios ficheros de código HULK (con extensión ‘.hulk’).

Para compilar y ejecutar un fichero, usa los comandos antes mencionados. Es importante que el fichero de código HULK esté en el directorio del proyecto y que haya uno solo con esa extensión.

3 Sintaxis del Lenguaje HULK

A continuación se presenta un resumen de la sintaxis del lenguaje HULK.

3.1 Estructura General

Un programa en HULK consiste en una secuencia de declaraciones y expresiones. Las expresiones deben terminar con un punto y coma (;).

```
// Ejemplo de un programa simple
let a = 42 in print(a);
```

3.2 Tipos de Datos

HULK soporta los siguientes tipos de datos básicos:

- **Number:** Números, tanto enteros como decimales.
- **String:** Cadenas de texto.
- **Boolean:** Valores ‘true’ o ‘false’.

También soporta la definición de nuevos tipos (clases) con herencia simple.

4 Arquitectura del Compilador

El compilador de HULK se compone de varias fases: análisis léxico, análisis sintáctico, análisis semántico y generación de código. A continuación se describe brevemente cada una de estas fases y su implementación.

4.1 Análisis Léxico con Flex

El analizador léxico se encarga de convertir el texto de entrada en una secuencia de tokens que representan las unidades mínimas de significado en el lenguaje. En el compilador HULK, esta fase se implementa utilizando Flex, un generador de analizadores léxicos.

4.1.1 Funcionamiento de Flex

Flex toma como entrada un archivo de definiciones léxicas (‘lexer.l’) que contiene patrones (expresiones regulares) y acciones asociadas. Al compilar este archivo, Flex genera un programa en C/C++ (‘lex.yy.c’) que implementa un autómata finito determinista para reconocer los tokens.

En ‘lexer.l’, cada token se define mediante:

- Una expresión regular que describe el patrón del token
- El código a ejecutar cuando se reconoce ese patrón

Ejemplo de definición de tokens en HULK:

```
"true"      { yylval.boolean = true; return BOOLEAN; }
"false"     { yylval.boolean = false; return BOOLEAN; }
"function"  { return FUNCTION_; }
"let"       { return LET; }
[a-zA-Z_][a-zA-Z0-9]* { yylval.str = strdup(yytext); return ID_; }
[0-9]+      { yylval.num = atoi(yytext); return NUMBER; }
```

4.2 Análisis Sintáctico con Bison

El analizador sintáctico verifica que la secuencia de tokens siga las reglas gramaticales del lenguaje. En HULK, esta fase utiliza Bison, un generador de parsers LALR(1).

4.2.1 Funcionamiento de Bison

Bison toma como entrada un archivo de definición de gramática (‘parser.y’) que contiene:

- Declaraciones de tokens y tipos
- Reglas de producción de la gramática
- Acciones semánticas asociadas a cada regla

A partir de esta definición, Bison genera un analizador en C/C++ (‘parser.tab.c’ y ‘parser.tab.h’) que implementa un autómata con pila para reconocer la estructura sintáctica del programa.

La gramática completa de HULK, tal como está definida en el archivo ‘parser.y’, se encuentra en el Apéndice A. Esta gramática define todas las construcciones sintácticas del lenguaje, junto con las acciones semánticas que construyen el AST correspondiente.

4.3 Árbol de Sintaxis Abstracta (AST)

Las acciones semánticas en Bison construyen un Árbol de Sintaxis Abstracta (AST), que es una representación jerárquica de la estructura del programa. Cada nodo del árbol representa un constructo del lenguaje (expresiones, sentencias, declaraciones, etc.).

En HULK, cada tipo de nodo se implementa como una clase derivada de la clase base ‘ASTNode’. Por ejemplo, ‘BinOpNode’ para operaciones binarias, ‘FloatNode’ para constantes numéricas, etc.

4.4 Análisis Semántico

El análisis semántico verifica que el programa tenga sentido según las reglas del lenguaje: comprobación de tipos, existencia de variables y funciones, etc. En HULK, este análisis se implementa utilizando el patrón Visitor sobre el AST.

El análisis se realiza en varias fases:

4.4.1 Recolector de Tipos

Implementado en ‘type_collector_visitor.cpp’, recorre el AST para registrar todos los tipos definidos por el usuario.

4.4.2 Recolector de Símbolos

Implementado en ‘symbol_collector_visitor.cpp’, recorre el AST para registrar todas las declaraciones de variables y funciones.

4.4.3 Verificador Semántico

Implementado como parte del visitor principal, verifica que las operaciones sean semánticamente válidas: tipos compatibles en operaciones, variables y funciones declaradas antes de usarse, etc.

4.5 Generación de Código

La fase final del compilador transforma el AST en código ejecutable. HULK utiliza LLVM (Low Level Virtual Machine) como backend para la generación de código.

4.5.1 LLVM y JIT

LLVM proporciona una representación intermedia (IR) del código, que luego puede ser optimizada y traducida a código máquina para diferentes arquitecturas. El compilador de HULK utiliza el compilador JIT (Just-In-Time) de LLVM para generar y ejecutar código directamente en memoria.

El proceso de generación de código se implementa en:

- ‘visitor.cpp’ en el directorio ‘codegen/’: Visitante que recorre el AST y genera el IR de LLVM
- ‘jit.cpp’: Clase que gestiona la compilación JIT y la ejecución del código

El visitor de generación de código traduce cada nodo del AST a instrucciones LLVM IR. Por ejemplo, un nodo ‘BinOpNode’ se traduce en la correspondiente instrucción aritmética de LLVM, mientras que un nodo ‘IfNode’ se traduce en bloques básicos con saltos condicionales.

5 Ejemplos de Código

5.1 Hola Mundo

```
print("Hello, World!");
```

5.2 Declaración de Variables y Expresiones

```
{
  let message = "El resultado es: " in {
    let a = 10 , b = 32 in print(message @@ (a + b));
  };
}
```

5.3 Definición de Funciones

```
{
  function add(a: Number, b: Number): Number => a + b;

  print(add(5, 7));
}
```

5.4 Estructuras Condicionales

```
{
  let a = 10 in if (a > 5) {
    print("a es mayor que 5");
  } else {
    print("a no es mayor que 5");
  };
}
```

5.5 Definición de Tipos

```
{
  type Point {
    x = 0;
    y = 0;

    setX(x: Number) => self.x = x;
    setY(y: Number) => self.y = y;
  }

  let p1 = new Point() in {
    p1.setX(5);
    p1.setY(6);
    let p2 = new Point() in {
      p2.setX(3);
      p2.setY(4);
      let dx = p1.x - p2.x in {
        let dy = p1.y - p2.y in {
          print(sqrt(dx * dx + dy * dy));
        };
      };
    };
  };
}
```

6 Resolución de Problemas

6.1 Errores Comunes

- **Errores léxicos:** Se producen cuando el programa contiene caracteres o construcciones que no son reconocidos como tokens. El mensaje de error indicará la línea y el carácter no reconocido.

- **Errores sintácticos:** Se producen cuando la estructura del programa no sigue las reglas gramaticales. El mensaje indicará la línea y el token que no encaja en la gramática.
- **Errores semánticos:** Incluyen el uso de variables no declaradas, tipos incompatibles, etc. El mensaje de error intentará ser informativo sobre la naturaleza del problema.

7 Conclusión

El compilador HULK implementa un lenguaje de programación funcional-imperativo con tipado estático. Utiliza herramientas estándar como Flex y Bison para las fases iniciales, y LLVM para la generación de código eficiente. El diseño orientado a objetos del compilador, con el extensivo uso del patrón Visitor, facilita su mantenimiento y extensión.

Este manual proporciona la información básica para utilizar el compilador y comprender su funcionamiento interno, pero no reemplaza la lectura del código fuente para entender los detalles de la implementación.

8 Implementaciones Alternativas

Durante el desarrollo del proyecto, se implementaron versiones propias del analizador léxico y sintáctico para un control más granular del proceso de compilación. Aunque finalmente se optó por utilizar Flex y Bison en la versión principal del compilador debido a su robustez y eficiencia, estas implementaciones alternativas siguen siendo parte del proyecto y pueden ser utilizadas para fines académicos o experimentales.

8.1 Lexer Personalizado

El lexer personalizado, implementado en C++, se basa en la teoría de autómatas finitos.

8.1.1 Características Principales

- **Definición programática de tokens:** Los tokens y sus patrones se definen directamente en C++ en el archivo `hulkGrammar.hpp`.
- **Uso de expresiones regulares:** Cada token se define mediante una expresión regular que determina los patrones de caracteres que reconoce.
- **Construcción de autómatas:** Las expresiones regulares se convierten en Autómatas Finitos No Deterministas (NFA) y luego en Autómatas Finitos Deterministas (DFA) para un reconocimiento eficiente de tokens.
- **Manejo detallado de errores:** Proporciona información precisa sobre errores léxicos, incluyendo la posición y descripción del error.

8.1.2 Implementación

La implementación se encuentra principalmente en el directorio `Automata/`, que contiene las clases para representar y manipular autómatas finitos:

- `nfa.cpp/h`: Implementación de Autómatas Finitos No Deterministas
- `dfa.cpp/h`: Implementación de Autómatas Finitos Deterministas
- `operations/operations.cpp/h`: Operaciones sobre autómatas (unión, concatenación, clausura de Kleene, etc.)
- `utils/ContainerSet.cpp/h`: Estructuras de datos para conjuntos utilizados en los algoritmos de autómatas

Las expresiones regulares y definiciones de tokens se establecen programáticamente en el archivo `hulkGrammar.hpp`.

8.2 Parser SLR1 Personalizado

El parser SLR1 (Simple LR) implementado manualmente proporciona un control completo sobre el proceso de análisis sintáctico, permitiendo personalizar aspectos como la construcción de la tabla de parsing y las acciones semánticas.

8.2.1 Características Principales

- **Gramática programable:** La gramática se define directamente en código C++, permitiendo una fácil modificación y experimentación.
- **Cálculo automático de conjuntos FIRST y FOLLOW:** El parser calcula automáticamente los conjuntos FIRST y FOLLOW necesarios para la construcción de la tabla SLR(1).
- **Construcción de la tabla de parsing:** Genera las tablas de acción y goto requeridas por el algoritmo LR.
- **Manejo personalizado de errores sintácticos:** Proporciona mensajes de error detallados, incluyendo los tokens esperados en cada punto.
- **Acciones semánticas flexibles:** Las acciones semánticas se definen como funciones lambda asociadas a cada producción, permitiendo una construcción muy personalizada del AST.

8.2.2 Implementación

La implementación se encuentra principalmente en los directorios `Grammar/` y `Parser/`:

- `Grammar/grammar.cpp/h`: Definición y manejo de gramáticas libres de contexto
- `Grammar/symbol.cpp/h`: Representación de símbolos terminales y no terminales
- `Grammar/production.cpp/h`: Representación de producciones gramaticales
- `Parser/SLR1Parser.cpp/h`: Implementación del algoritmo de parsing SLR(1)
- `Parser/Item.cpp/h`: Representación de ítems LR para la construcción de estados

La definición de la gramática de HULK para el parser SLR1 se encuentra en el archivo `hulkGrammar.hpp`, donde cada producción se define utilizando la función `AddProduction` con `AttrProd` (producciones con atributos). Las producciones se definen utilizando una sintaxis clara y declarativa donde las acciones semánticas se asocian directamente a las producciones:

```
// Ejemplo: input -> line
g.AddProduction(AttrProd(input, Sentence(line), [](const std::vector<ElementType>& args) -> ElementT
    ASTNode* line_node = std::get<ASTNode*>(args[0]);
    return line_node;
));

// Ejemplo: lines_block -> { lines }
g.AddProduction(AttrProd(lines_block, Sentence({LKEY, lines, RKEY}), [](const std::vector<ElementTyp
    BlockNode* lines_node = static_cast<BlockNode*>(std::get<ASTNode*>(args[1]));
    return lines_node;
));
```

8.2.3 Estado Actual de la Implementación

Es importante destacar que la gramática definida en `hulkGrammar.hpp` es un trabajo en progreso. Este archivo representa un intento de transcribir la gramática definida en `parser.y` (utilizada por Bison) para su uso con el parser LALR1 personalizado que aún está en desarrollo. La implementación está incompleta y requiere revisión antes de poder ser utilizada como sustituto completo del parser generado por Bison e incluso del parser SLR1 personalizado.

La versión actual de `hulkGrammar.hpp` sirve principalmente como demostración de las capacidades del framework de parsing desarrollado, mostrando cómo se pueden definir gramáticas programáticamente

con acciones semánticas integradas, pero no debe considerarse una implementación completa y funcional de la gramática de HULK.

Este enfoque permite una gran flexibilidad en la construcción del AST y facilita la experimentación con diferentes gramáticas y estrategias de análisis sintáctico.

A Gramática Completa de HULK en Bison

A continuación se presenta la gramática completa del lenguaje HULK tal como está definida en el archivo `parser.y`. La gramática incluye las acciones semánticas que construyen el AST, pero se han omitido las declaraciones iniciales y las definiciones de tokens para mayor claridad.

```
input:
    line { root = $1; }
    | lines_block { root = $1; }
    ;

lines_block:
    LKEY lines RKEY { $$ = $2; }
    ;

lines:
    /* empty */ { $$ = new BlockNode({}, yylineno); }
    | non_empty_lines { $$ = $1; }
    ;

non_empty_lines:
    line { $$ = new BlockNode({$1}, yylineno); }
    | non_empty_lines line { $1->add_child($2); $$ = $1; }
    ;

line:
    expr SEMICOLON { $$ = $1; }
    | func_asign SEMICOLON { $$ = $1; }
    | type_node_decl { $$ = $1; }
    ;

expr:
    arit_op { $$ = $1; }
    | bool_expr { $$ = $1; }
    | WHILE while_expr { $$ = $2; }
    | STRING { $$ = new StringNode($1, yylineno); }
    | id_expr %prec ASS_DES { $$ = $1; }
    | func_call { $$ = $1; }
    | let_assign { $$ = $1; }
    | IF conditional { $$ = $2; }
    | member_access_expr { $$ = $1; }
    | expr ASS_DES expr { $$ = new BinOpNode($1, ":", $3, yylineno); }
    | expr ASSIGN expr { $$ = new BinOpNode($1, "=", $3, yylineno); }
    | expr ARROBA_ expr { $$ = new BinOpNode($1, "@", $3, yylineno); }
    | expr D_ARROBA_ expr { $$ = new BinOpNode($1, "@@", $3, yylineno); }
    ;

func_asign:
    FUNCTION_ ID_ LPARENT args_list RPARENT INLINE expr { $$ = new AssignFuncNode(new IDNode($2, yylin
    | FUNCTION_ ID_ LPARENT args_list RPARENT TWOPOINTS ID_ INLINE expr { $$ = new AssignFuncNode(new I
    | FUNCTION_ ID_ LPARENT args_list RPARENT LKEY lines RKEY { $$ = new AssignFuncNode(new IDNode($2,
    | FUNCTION_ ID_ LPARENT args_list RPARENT TWOPOINTS ID_ LKEY lines RKEY { $$ = new AssignFuncNode(n
    ;
```

```

args_list:
/* empty */ { $$ = new ArgsList({}, yylineno); }
| id_expr { $$ = new ArgsList({$1}, yylineno); }
| args_list COLON id_expr { $1->add_child($3); $$ = $1; }
;

id_expr:
ID_ TWOPOINTS ID_ { $$ = new IDNode($1, $3, yylineno); }
| ID_ { $$ = new IDNode($1, yylineno); }
;

let_assign:
LET var_assign_list IN expr { $$ = new LetAssign($2->assigns, $4, yylineno); }
| LET var_assign_list IN lines_block { $$ = new LetAssign($2->assigns, $4, yylineno); }
;

var_assign_list:
id_expr ASSIGN expr { $$ = new VarAssignList({ new VarAssign($1, $3, yylineno) }, yylineno); }
| id_expr ASSIGN expr AS_ ID_ { $$ = new VarAssignList({ new VarAssign($1, $3, $5, yylineno)}, yyli
| id_expr ASSIGN new_expr { $$ = new VarAssignList({ new VarAssign($1, $3, yylineno)}, yylineno); }
| var_assign_list COLON id_expr ASSIGN expr { $1->add_child(new VarAssign($3, $5, yylineno)); $$ =
| var_assign_list COLON id_expr ASSIGN expr AS_ ID_ { $1->add_child(new VarAssign($3, $5, $7, yylin
| var_assign_list COLON id_expr ASSIGN new_expr { $1->add_child(new VarAssign($3, $5, yylineno)); $
;

new_expr:
NEW ID_ LPARENT expr_list RPARENT { $$ = new NewTypeNode($2, $4->children, yylineno); }
;

expr_list:
/* empty */ { $$ = new ASTNodeVector({}, yylineno); }
| expr { $$ = new ASTNodeVector({$1}, yylineno); }
| new_expr { $$ = new ASTNodeVector({$1}, yylineno); }
| expr_list COLON expr { $1->add_child($3); $$ = $1; }
| expr_list COLON new_expr { $1->add_child($3); $$ = $1; }
;

func_call:
ID_ LPARENT expr_list RPARENT { $$ = new FunctionCallNode($1, $3, yylineno); }
;

arit_op:
NUMBER { $$ = new FloatNode($1, yylineno); }
| LPARENT expr RPARENT { $$ = $2; }
| expr PLUS expr { $$ = new BinOpNode( $1, "+", $3, yylineno); }
| expr MINUS expr { $$ = new BinOpNode( $1, "-", $3, yylineno); }
| expr TIMES expr { $$ = new BinOpNode( $1, "*", $3, yylineno); }
| expr DIV expr { $$ = new BinOpNode( $1, "/", $3, yylineno); }
| expr POW expr { $$ = new BinOpNode( $1, "^", $3, yylineno); }
| MINUS expr %prec UMINUS { $$ = new UnaryOpNode("-", $2, yylineno); }
;

bool_expr:
BOOLEAN { $$ = new BoolExprNode(new BoolNode($1, yylineno), yylineno); }
| expr GREATER_EQUAL expr { $$ = new BoolExprNode(new BinOpNode($1, ">=", $3, yylineno), yylineno); }
| expr GREATER expr { $$ = new BoolExprNode(new BinOpNode($1, ">", $3, yylineno), yylineno); }
| expr LESS_EQUAL expr { $$ = new BoolExprNode(new BinOpNode($1, "<=", $3, yylineno), yylineno); }

```

```

| expr LESS expr { $$ = new BoolExprNode(new BinOpNode($1, "<", $3, yylineno), yylineno); }
| expr EQUAL expr { $$ = new BoolExprNode(new BinOpNode($1, "==", $3, yylineno), yylineno); }
| expr DISTINCT expr { $$ = new BoolExprNode(new BinOpNode($1, "!=", $3, yylineno), yylineno); }
| expr AND_ expr { $$ = new BoolExprNode(new BinOpNode($1, "&", $3, yylineno), yylineno); }
| expr OR_ expr { $$ = new BoolExprNode(new BinOpNode($1, "|", $3, yylineno), yylineno); }
| NOT_ expr { $$ = new BoolExprNode(new UnaryOpNode("!", $2, yylineno), yylineno); }
| id_expr IS ID_ { $$ = new BoolExprNode(new BinOpNode($1, "is", new IDNode($3, yylineno), yylineno); }
| func_call IS ID_ { $$ = new BoolExprNode(new BinOpNode($1, "is", new IDNode($3, yylineno), yylineno); }
;

conditional:
LPARENT bool_expr RPARENT expr ELSE expr { $$ = new Conditional($2, $4, $6, yylineno); }
| LPARENT bool_expr RPARENT lines_block ELSE expr { $$ = new Conditional($2, $4, $6, yylineno); }
| LPARENT bool_expr RPARENT expr ELSE lines_block { $$ = new Conditional($2, $4, $6, yylineno); }
| LPARENT bool_expr RPARENT lines_block ELSE lines_block { $$ = new Conditional($2, $4, $6, yylineno); }
| LPARENT bool_expr RPARENT expr ELIF conditional { $$ = new Conditional($2, $4, $6, yylineno); }
| LPARENT bool_expr RPARENT lines_block ELIF conditional { $$ = new Conditional($2, $4, $6, yylineno); }
;

while_expr:
LPARENT bool_expr RPARENT lines_block { $$ = new WhileNode($2, $4, yylineno); }
| LPARENT bool_expr RPARENT expr SEMICOLON { $$ = new WhileNode($2, $4, yylineno); }
;

type_node_decl:
TYPE ID_ LKEY type_body_elements RKEY { $$ = new TypeDeclNode(new IDNode($2, yylineno), new ArgsList($3, yylineno)); }
| TYPE ID_ INHERITS ID_ LKEY type_body_elements RKEY { $$ = new TypeDeclNode(new IDNode($2, yylineno), new IDNode($3, yylineno)); }
| TYPE ID_ INHERITS ID_ LPARENT args_list RPARENT LKEY type_body_elements RKEY { $$ = new TypeDeclNode(new IDNode($2, yylineno), new IDNode($3, yylineno)); }
| TYPE ID_ LPARENT args_list RPARENT LKEY type_body_elements RKEY { $$ = new TypeDeclNode(new IDNode($2, yylineno), new IDNode($3, yylineno)); }
| TYPE ID_ LPARENT args_list RPARENT INHERITS ID_ LKEY type_body_elements RKEY { $$ = new TypeDeclNode(new IDNode($2, yylineno), new IDNode($3, yylineno)); }
| TYPE ID_ LPARENT args_list RPARENT INHERITS ID_ LPARENT args_list RPARENT LKEY type_body_elements RKEY { $$ = new TypeDeclNode(new IDNode($2, yylineno), new IDNode($3, yylineno)); }
;

type_body_elements:
/* empty */ { $$ = new ASTNodeVector({}, yylineno); }
| type_body_elements attribute { $1->add_child($2); $$ = $1; }
| type_body_elements method { $1->add_child($2); $$ = $1; }
;

attribute:
id_expr ASSIGN expr SEMICOLON { $$ = new VarAssign($1, $3, yylineno); }
| id_expr ASSIGN expr AS_ ID_ SEMICOLON { $$ = new VarAssign($1, $3, $5, yylineno); }
;

method:
ID_ LPARENT args_list RPARENT INLINE expr SEMICOLON { $$ = new AssignFuncNode(new IDNode($1, yylineno), new IDNode($3, yylineno)); }
| ID_ LPARENT args_list RPARENT TWOPOINTS ID_ INLINE expr SEMICOLON { $$ = new AssignFuncNode(new IDNode($1, yylineno), new IDNode($3, yylineno)); }
| ID_ LPARENT args_list RPARENT LKEY lines RKEY { $$ = new AssignFuncNode(new IDNode($1, yylineno), new IDNode($3, yylineno)); }
| ID_ LPARENT args_list RPARENT TWOPOINTS ID_ LKEY lines RKEY { $$ = new AssignFuncNode(new IDNode($1, yylineno), new IDNode($3, yylineno)); }
;

member_access_expr:
ID_ ACCESS ID_ { $$ = new AccessNode($1, new AttributeMember($3, yylineno), yylineno); }
| ID_ ACCESS ID_ LPARENT expr_list RPARENT { $$ = new AccessNode($1, new MethodMember($3, $5->child(1), yylineno), yylineno); }
;

```

B Notas sobre la Gramática

La gramática del lenguaje HULK se ha diseñado con varias características importantes:

- **Soporte para programación funcional:** Con construcciones como expresiones ‘let-in’ y funciones anónimas.
- **Soporte para programación orientada a objetos:** Con definiciones de tipos (clases), herencia y acceso a miembros.
- **Expresiones condicionales:** Estructuras ‘if-else’ y ‘if-elif-else’ para control de flujo.
- **Bucles:** Construcciones ‘while’ para iteración.
- **Operadores sobrecargados:** Para operaciones aritméticas, lógicas y de concatenación (‘@’ y ‘@@’).

Las acciones semánticas (código entre llaves) son responsables de construir el AST mientras se analiza la estructura sintáctica del programa.